In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.io import arff

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

In [2]:
```
file_path = "3year.arff"

data, meta = arff.loadarff(file_path)

df = pd.DataFrame(data)

print("=" * 70)
print("POLISH COMPANIES BANKRUPTCY DATASET")
print("=" * 70)

print("Dataset Shape:", df.shape)

print("\nFirst 5 rows:")
print(df.head())
```

```
================================================================
==
POLISH COMPANIES BANKRUPTCY DATASET
================================================================
==
Dataset Shape: (10503, 65)

First 5 rows:
      Attr1    Attr2    Attr3   Attr4    Attr5    Attr6     Attr7
Attr8  \
0  0.174190  0.41299  0.14371  1.3480 -28.9820  0.60383  0.219460
1.1225
1  0.146240  0.46038  0.28230  1.6294   2.5952  0.00000  0.171850
1.1721
2  0.000595  0.22612  0.48839  3.1599  84.8740  0.19114  0.004572
2.9881
3  0.024526  0.43236  0.27546  1.7833 -10.1050  0.56944  0.024526
1.3057
4  0.188290  0.41504  0.34231  1.9279 -58.2740  0.00000  0.233580
1.4094

    Attr9   Attr10  ...    Attr56    Attr57   Attr58    Attr59  Attr
60  \
0  1.1961  0.46359  ...  0.163960  0.375740  0.83604  0.000007  9.71
45
1  1.6018  0.53962  ...  0.027516  0.271000  0.90108  0.000000  5.98
82
2  1.0077  0.67566  ...  0.007639  0.000881  0.99236  0.000000  6.77
42
3  1.0509  0.56453  ...  0.048398  0.043445  0.95160  0.142980  4.22
86
4  1.3393  0.58496  ...  0.176480  0.321880  0.82635  0.073039  2.59
12

   Attr61   Attr62  Attr63  Attr64  class
0  6.2813   84.291  4.3303  4.0341   b'0'
1  4.1103  102.190  3.5716  5.9500   b'0'
2  3.7922   64.846  5.6287  4.4581   b'0'
3  5.0528   98.783  3.6950  3.4844   b'0'
4  7.0756  100.540  3.6303  4.6375   b'0'

[5 rows x 65 columns]
```

In [3]:

```python
# ============================================================
# DECODE BYTE / STRING COLUMNS
# ============================================================

for column in df.columns:

    if df[column].dtype == object:

        df[column] = df[column].apply(
            lambda x: x.decode("utf-8")
            if isinstance(x, bytes)
            else x
        )
```

```
print("Byte/string conversion completed.")

print("\nData Types:")
print(df.dtypes)
```

```
Byte/string conversion completed.

Data Types:
Attr1     float64
Attr2     float64
Attr3     float64
Attr4     float64
Attr5     float64
           ...
Attr61    float64
Attr62    float64
Attr63    float64
Attr64    float64
class      object
Length: 65, dtype: object
```

In [4]:
```
# ================================================================
# DATASET INFORMATION
# ================================================================

print("=" * 70)
print("DATASET INFORMATION")
print("=" * 70)

print("\nShape:")
print(df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes.value_counts())

print("\nMissing Values:")
print(df.isnull().sum().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())
```

```
==========================================================================
==
DATASET INFORMATION
==========================================================================
==

Shape:
(10503, 65)

Columns:
['Attr1', 'Attr2', 'Attr3', 'Attr4', 'Attr5', 'Attr6', 'Attr7', 'Att
r8', 'Attr9', 'Attr10', 'Attr11', 'Attr12', 'Attr13', 'Attr14', 'Att
r15', 'Attr16', 'Attr17', 'Attr18', 'Attr19', 'Attr20', 'Attr21', 'A
ttr22', 'Attr23', 'Attr24', 'Attr25', 'Attr26', 'Attr27', 'Attr28',
'Attr29', 'Attr30', 'Attr31', 'Attr32', 'Attr33', 'Attr34', 'Attr3
5', 'Attr36', 'Attr37', 'Attr38', 'Attr39', 'Attr40', 'Attr41', 'Att
r42', 'Attr43', 'Attr44', 'Attr45', 'Attr46', 'Attr47', 'Attr48', 'A
ttr49', 'Attr50', 'Attr51', 'Attr52', 'Attr53', 'Attr54', 'Attr55',
'Attr56', 'Attr57', 'Attr58', 'Attr59', 'Attr60', 'Attr61', 'Attr6
2', 'Attr63', 'Attr64', 'class']

Data Types:
float64    64
object      1
Name: count, dtype: int64

Missing Values:
9888

Duplicate Rows:
87
```

In [5]:
```
# ============================================================
# IDENTIFY TARGET
# ============================================================

target_column = df.columns[-1]

print("Target column:", target_column)

print("\nTarget values:")
print(df[target_column].value_counts())
```

```
Target column: class

Target values:
class
0    10008
1      495
Name: count, dtype: int64
```

In [6]:
```
# ============================================================
# DATASET INFORMATION
# ============================================================

print("=" * 70)
print("DATASET INFORMATION")
```

```
print("=" * 70)

print("\nShape:")
print(df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes.value_counts())

print("\nMissing Values:")
print(df.isnull().sum().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())
```

```
======================================================================
==
DATASET INFORMATION
======================================================================
==

Shape:
(10503, 65)

Columns:
['Attr1', 'Attr2', 'Attr3', 'Attr4', 'Attr5', 'Attr6', 'Attr7', 'Att
r8', 'Attr9', 'Attr10', 'Attr11', 'Attr12', 'Attr13', 'Attr14', 'Att
r15', 'Attr16', 'Attr17', 'Attr18', 'Attr19', 'Attr20', 'Attr21', 'A
ttr22', 'Attr23', 'Attr24', 'Attr25', 'Attr26', 'Attr27', 'Attr28',
'Attr29', 'Attr30', 'Attr31', 'Attr32', 'Attr33', 'Attr34', 'Attr3
5', 'Attr36', 'Attr37', 'Attr38', 'Attr39', 'Attr40', 'Attr41', 'Att
r42', 'Attr43', 'Attr44', 'Attr45', 'Attr46', 'Attr47', 'Attr48', 'A
ttr49', 'Attr50', 'Attr51', 'Attr52', 'Attr53', 'Attr54', 'Attr55',
'Attr56', 'Attr57', 'Attr58', 'Attr59', 'Attr60', 'Attr61', 'Attr6
2', 'Attr63', 'Attr64', 'class']

Data Types:
float64    64
object      1
Name: count, dtype: int64

Missing Values:
9888

Duplicate Rows:
87
```

In [7]:
```
# ============================================================
# IDENTIFY TARGET
# ============================================================

target_column = df.columns[-1]

print("Target column:", target_column)
```

```
print("\nTarget values:")
print(df[target_column].value_counts())
```

```
Target column: class

Target values:
class
0    10008
1      495
Name: count, dtype: int64
```

In [8]:
```
# ================================================================
# CONVERT FEATURES TO NUMERIC
# ================================================================

for column in df.columns:

    if column != target_column:

        df[column] = pd.to_numeric(
            df[column],
            errors="coerce"
        )

df[target_column] = pd.to_numeric(
    df[target_column],
    errors="coerce"
)

print("Numeric conversion completed.")

print(df.dtypes.tail())
```

```
Numeric conversion completed.
Attr61    float64
Attr62    float64
Attr63    float64
Attr64    float64
class       int64
dtype: object
```

In [9]:
```
# ================================================================
# REMOVE INVALID TARGET RECORDS
# ================================================================

df = df.dropna(
    subset=[target_column]
)

df[target_column] = df[target_column].astype(int)

print("Target distribution:")
print(df[target_column].value_counts())
```

```
Target distribution:
class
0    10008
1      495
Name: count, dtype: int64
```

In [10]:
```
# ============================================================
# MISSING VALUE ANALYSIS
# ============================================================

missing = df.isnull().sum()

missing = missing[
    missing > 0
].sort_values(
    ascending=False
)

print("Columns containing missing values:")

print(missing.head(20))
```

```
Columns containing missing values:
Attr37    4736
Attr21     807
Attr27     715
Attr60     592
Attr45     591
Attr54     228
Attr53     228
Attr28     228
Attr64     228
Attr24     227
Attr41     202
Attr32     101
Attr52      86
Attr47      86
Attr19      43
Attr39      43
Attr62      43
Attr56      43
Attr49      43
Attr13      43
dtype: int64
```

In [11]:
```
# ============================================================
# REMOVE DUPLICATES
# ============================================================

before = len(df)

df = df.drop_duplicates()

after = len(df)

print("Rows before removing duplicates:", before)
print("Rows after removing duplicates :", after)
```

```
print("Duplicates removed             :", before - after)
```

```
Rows before removing duplicates: 10503
Rows after removing duplicates : 10416
Duplicates removed             : 87
```

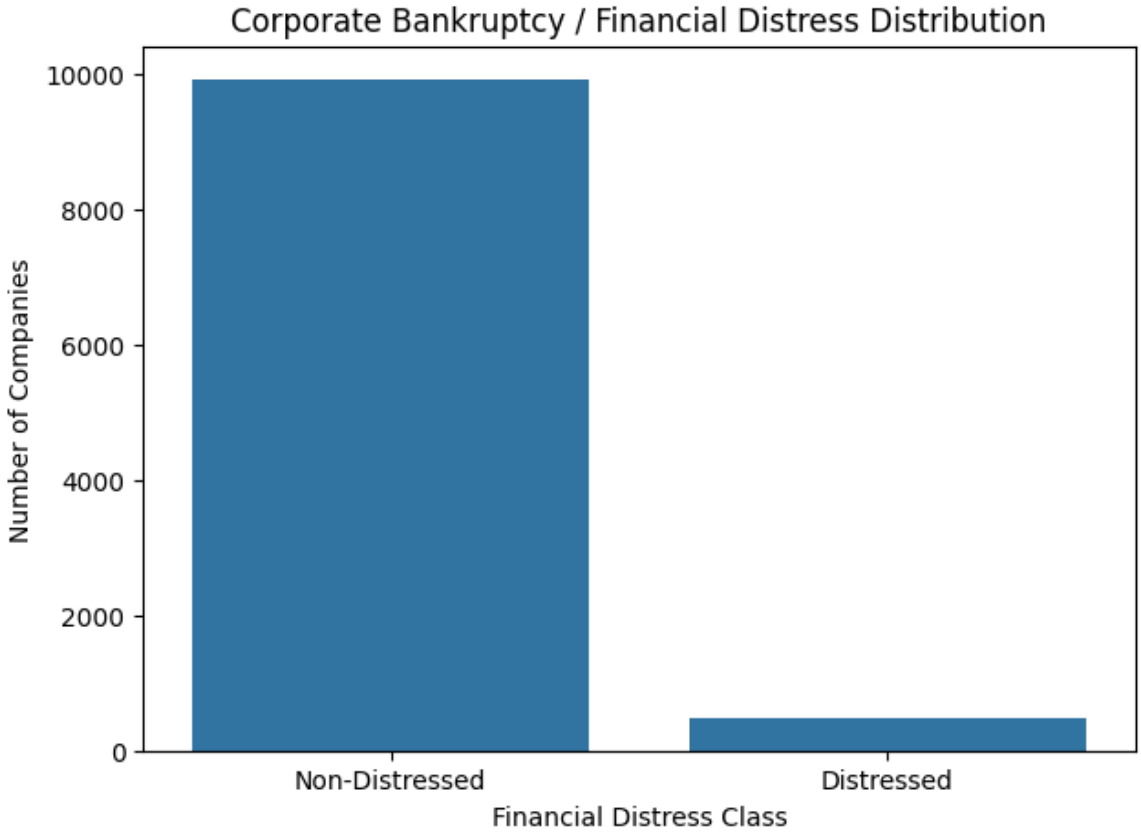
In [12]:
```
# ============================================================
# STATISTICAL SUMMARY
# ============================================================

print(
    df.describe().T
)
```

```
          count         mean             std           min         25%
\
Attr1   10416.0     0.052905        0.650356 -1.769200e+01    0.000665
Attr2   10416.0     0.621781        6.453747  0.000000e+00    0.254553
Attr3   10416.0     0.094550        6.446757 -4.797300e+02    0.017276
Attr4   10398.0    10.031805      525.877769  2.080200e-03    1.039425
Attr5   10391.0 -1359.332129   119075.917639 -1.190300e+07 -52.164000
...         ...          ...             ...           ...         ...
Attr61  10399.0    13.995539       84.048713 -6.590300e+00    4.491400
Attr62  10373.0   135.612155    26099.934909 -2.336500e+06   40.709000
Attr63  10398.0     9.119298       31.545909 -1.562200e-04    3.063025
Attr64  10191.0    35.996070      430.044997 -1.023000e-04    2.031050
class   10416.0     0.047331        0.212356  0.000000e+00    0.000000

              50%         75%          max
Attr1    0.043056    0.123867       52.652
Attr2    0.464850    0.690967      480.730
Attr3    0.198655    0.419793       17.708
Attr4    1.604900    2.958575    53433.000
Attr5    1.623400   56.464000   685440.000
...           ...         ...          ...
Attr61   6.686700   10.640500     4470.400
Attr62  70.626000  118.190000  1073500.000
Attr63   5.142800    8.900100     1974.500
Attr64   4.083600    9.720850    21499.000
class    0.000000    0.000000        1.000

[65 rows x 8 columns]
```

In [13]:
```
# ============================================================
# CLASS DISTRIBUTION
# ============================================================

class_counts = df[target_column].value_counts()

print("Class Distribution:")
print(class_counts)

plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x=target_column
```

```
)

plt.title("Corporate Bankruptcy / Financial Distress Distribution")

plt.xlabel("Financial Distress Class")

plt.ylabel("Number of Companies")

plt.xticks(
    [0, 1],
    ["Non-Distressed", "Distressed"]
)

plt.show()
```

```
Class Distribution:
class
0    9923
1     493
Name: count, dtype: int64
```

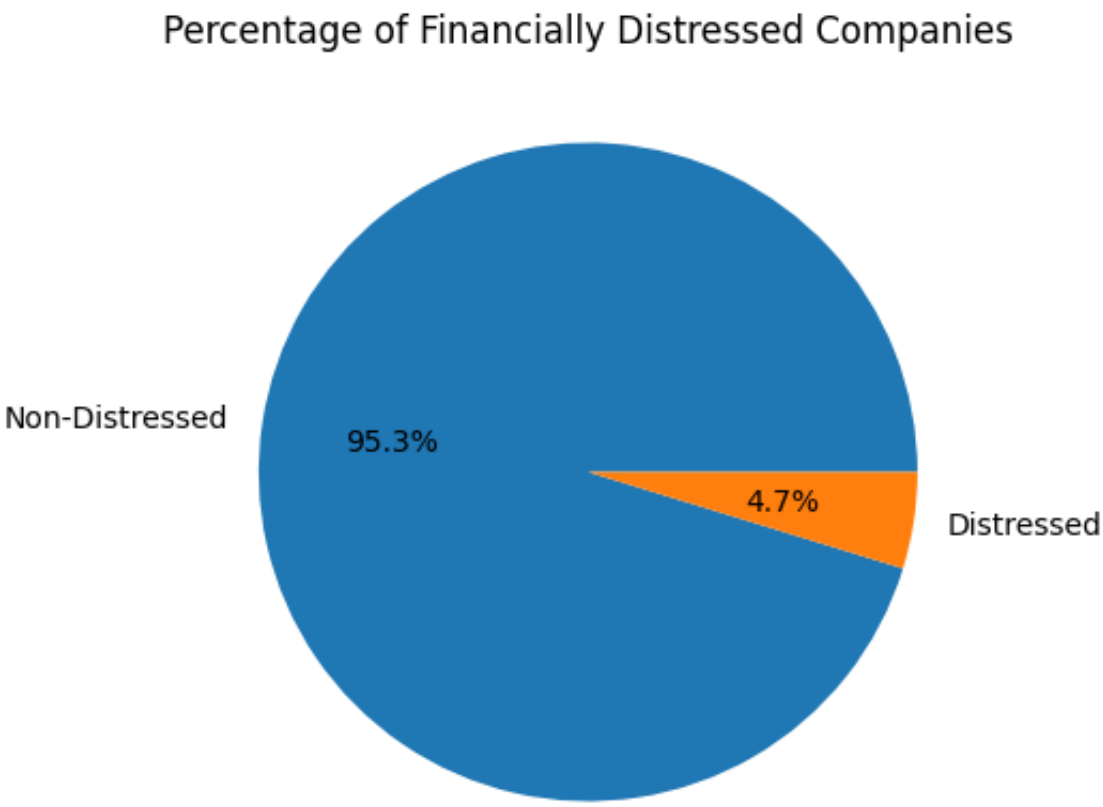


In [14]:

```
# ============================================================
# CLASS PERCENTAGE
# ============================================================

class_percentage = (
    df[target_column]
    .value_counts(normalize=True)
    * 100
)

print(
    class_percentage.round(2)
```

```
)

plt.figure(figsize=(7, 5))

plt.pie(
    class_percentage.values,
    labels=[
        "Non-Distressed",
        "Distressed"
    ],
    autopct="%1.1f%%"
)

plt.title(
    "Percentage of Financially Distressed Companies"
)

plt.show()
```

```
class
0    95.27
1     4.73
Name: proportion, dtype: float64
```



In [15]:

```
# ============================================================
# FINANCIAL RATIO STATISTICS
# ============================================================

numeric_features = df.drop(
    columns=[target_column]
)
```

```
print(
    numeric_features.describe().T.head(20)
)
```

```
          count         mean            std           min          2
5%  \
Attr1   10416.0     0.052905       0.650356 -1.769200e+01    0.00066
5
Attr2   10416.0     0.621781       6.453747  0.000000e+00    0.25455
3
Attr3   10416.0     0.094550       6.446757 -4.797300e+02    0.01727
6
Attr4   10398.0    10.031805     525.877769  2.080200e-03    1.03942
5
Attr5   10391.0 -1359.332129  119075.917639 -1.190300e+07  -52.16400
0
Attr6   10416.0    -0.123138       6.999619 -5.081200e+02    0.00000
0
Attr7   10416.0     0.065695       0.653707 -1.769200e+01    0.00210
5
Attr8   10402.0    16.418538     659.651634 -2.081800e+00    0.43005
0
Attr9   10413.0     1.824308       7.612845 -1.215700e+00    1.01170
0
Attr10  10416.0     0.364329       6.455319 -4.797300e+02    0.29671
5
Attr11  10416.0     0.086893       0.657991 -1.769200e+01    0.00988
0
Attr12  10398.0     2.391934     111.859463 -1.543800e+03    0.00616
9
Attr13  10373.0     0.373136      49.879957 -6.317100e+02    0.02062
2
Attr14  10416.0     0.065705       0.653706 -1.769200e+01    0.00211
2
Attr15  10408.0  3032.275358  109640.530193 -2.321800e+06  186.41000
0
Attr16  10402.0     2.710768     110.627442 -2.043000e+02    0.06026
0
Attr17  10402.0    17.791604     664.079527 -4.341100e-02    1.44645
0
Attr18  10416.0     0.070852       0.838686 -1.769200e+01    0.00211
2
Attr19  10373.0    -0.178400      11.241344 -7.716500e+02    0.00162
8
Attr20  10373.0    68.428538    1088.045393 -1.438600e-03   14.23200
0

              50%          75%           max
Attr1    0.043056     0.123867  5.265200e+01
Attr2    0.464850     0.690967  4.807300e+02
Attr3    0.198655     0.419793  1.770800e+01
Attr4    1.604900     2.958575  5.343300e+04
Attr5    1.623400    56.464000  6.854400e+05
Attr6    0.000000     0.070849  4.553300e+01
Attr7    0.051004     0.142415  5.265200e+01
```

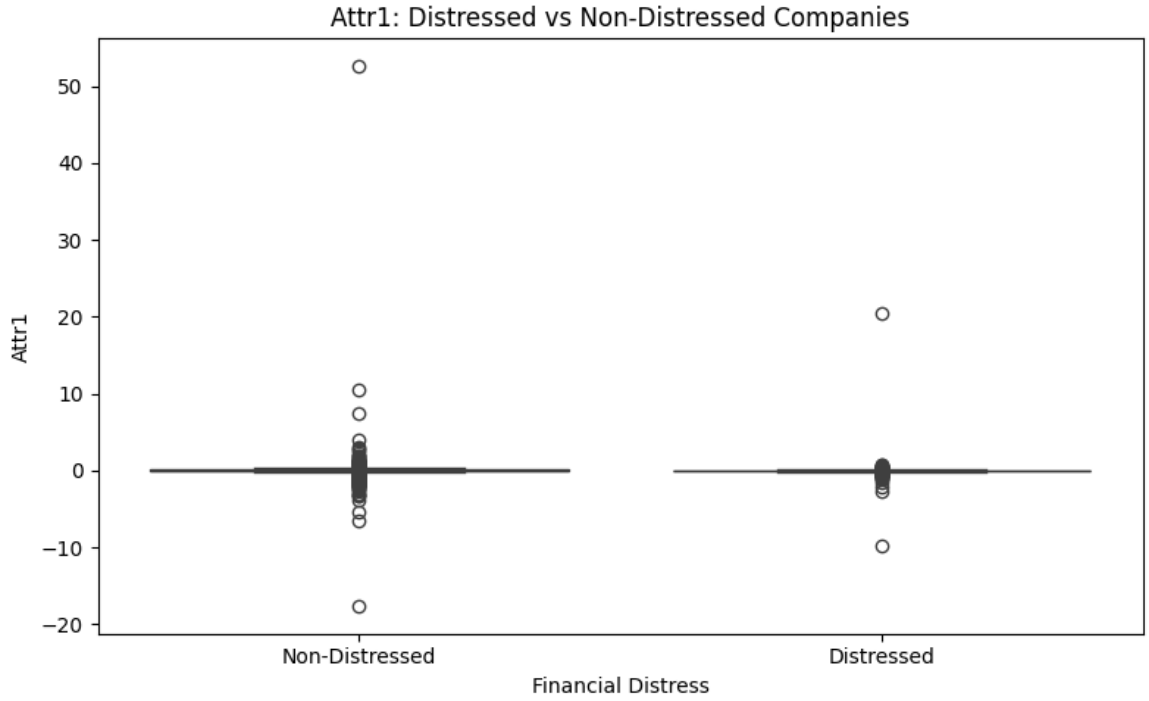
```
Attr8     1.108600     2.844075  5.343200e+04
Attr9     1.201600     2.068800  7.404400e+02
Attr10    0.514595     0.724762  1.183700e+01
Attr11    0.068218     0.161282  5.265200e+01
Attr12    0.155755     0.567560  8.259400e+03
Attr13    0.066283     0.133130  4.972000e+03
Attr14    0.051022     0.142415  5.265200e+01
Attr15  807.155000  2191.800000  1.023600e+07
Attr16    0.234800     0.642783  8.259400e+03
Attr17    2.148050     3.916350  5.343300e+04
Attr18    0.051022     0.142470  5.368900e+01
Attr19    0.032049     0.087458  1.239400e+02
Attr20   34.421000    63.658000  9.160000e+04
```

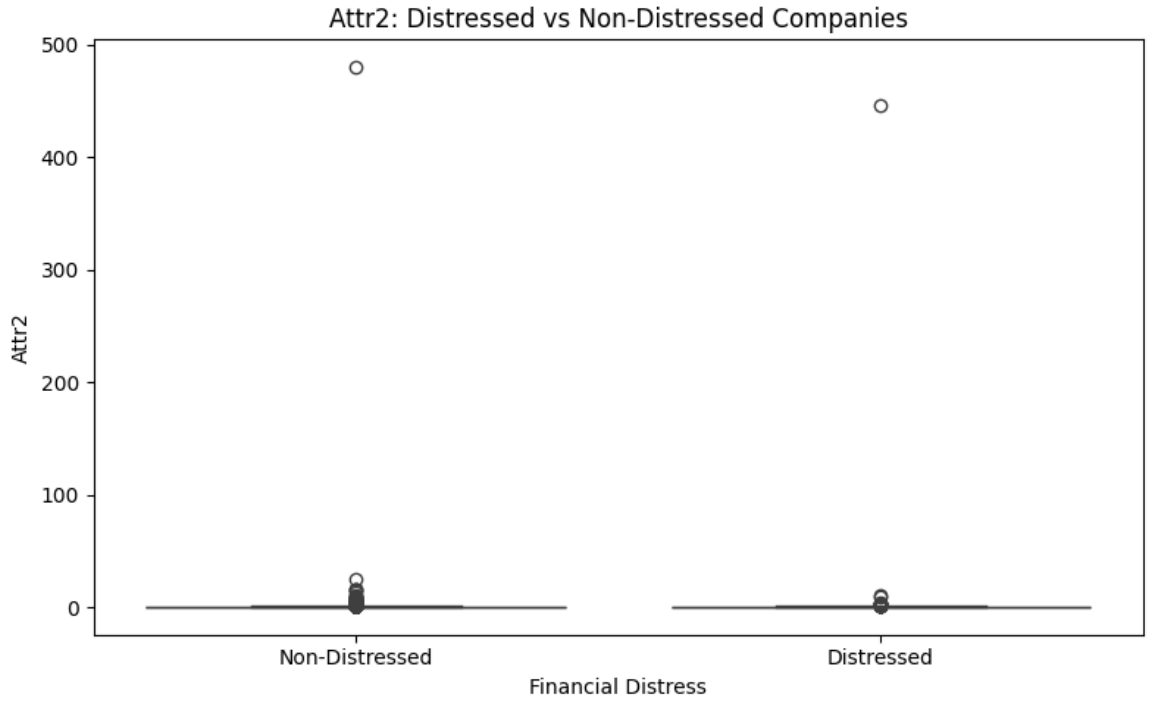
In [16]:
```
# ============================================================
# FINANCIAL RATIO COMPARISON
# ============================================================

selected_features = numeric_features.columns[:6]

print("Selected financial indicators:")
print(selected_features.tolist())
```

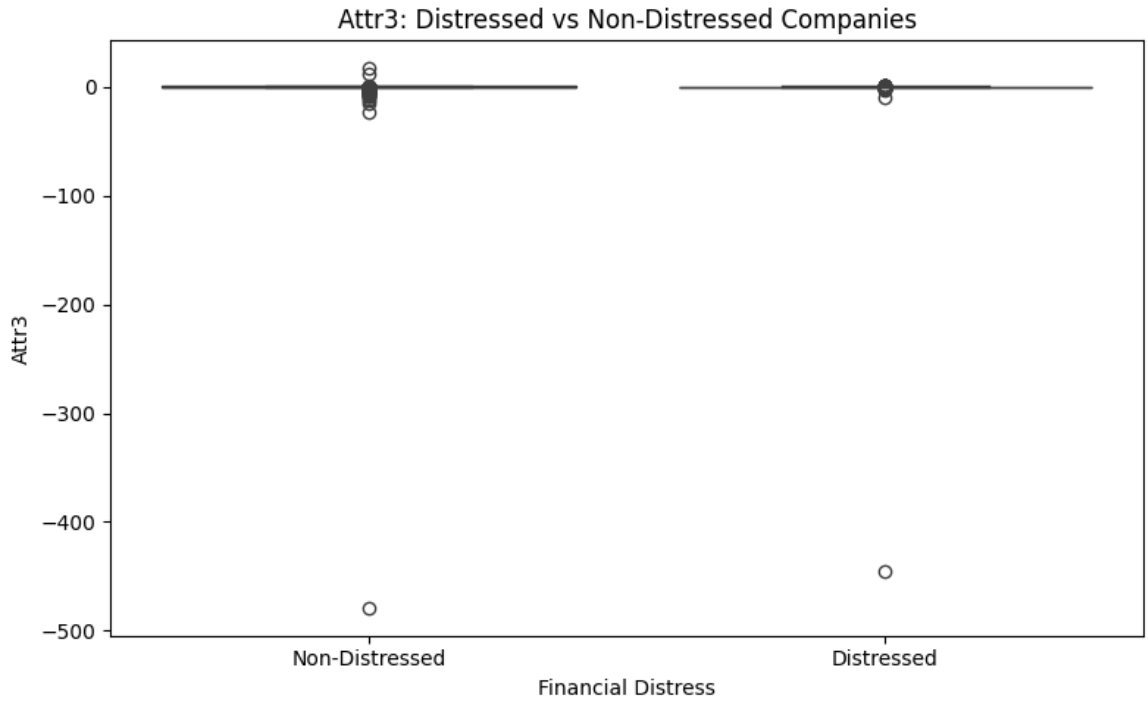
```
Selected financial indicators:
['Attr1', 'Attr2', 'Attr3', 'Attr4', 'Attr5', 'Attr6']
```

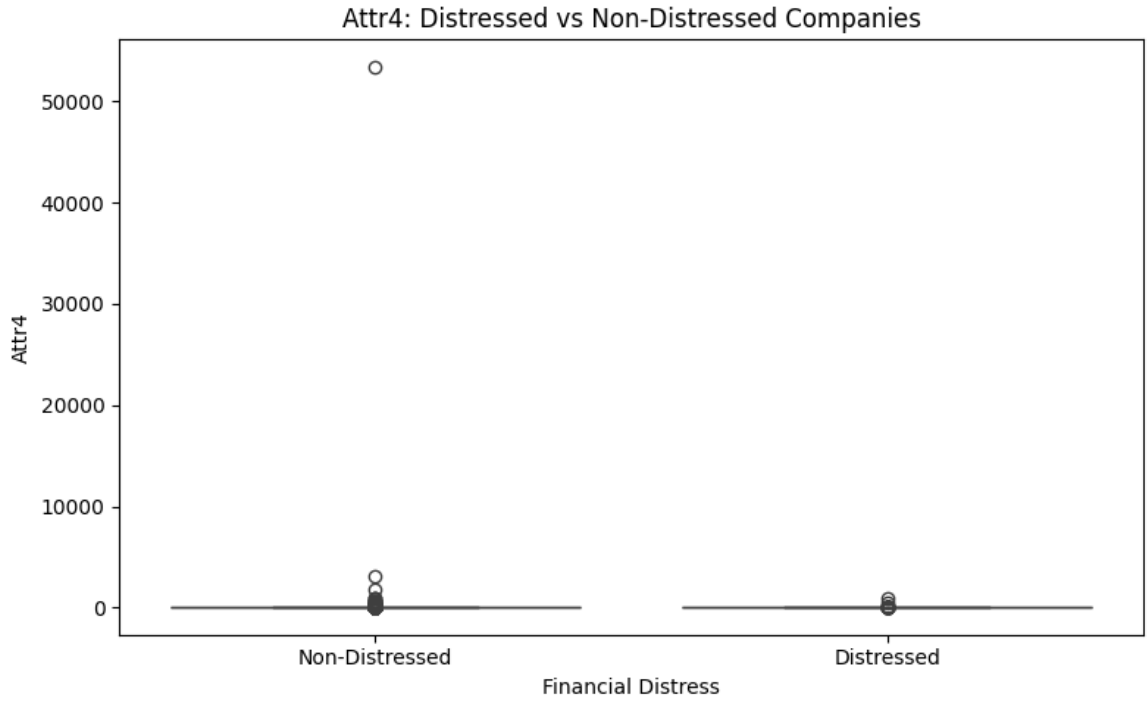
In [17]:
```
# ============================================================
# FINANCIAL RATIO BOXPLOTS
# ============================================================

for feature in selected_features:

    plt.figure(figsize=(8, 5))

    sns.boxplot(
        data=df,
        x=target_column,
        y=feature
    )

    plt.title(
        f"{feature}: Distressed vs Non-Distressed Companies"
    )

    plt.xlabel(
        "Financial Distress"
    )

    plt.ylabel(
        feature
    )

    plt.xticks(
        [0, 1],
        ["Non-Distressed", "Distressed"]
    )
```

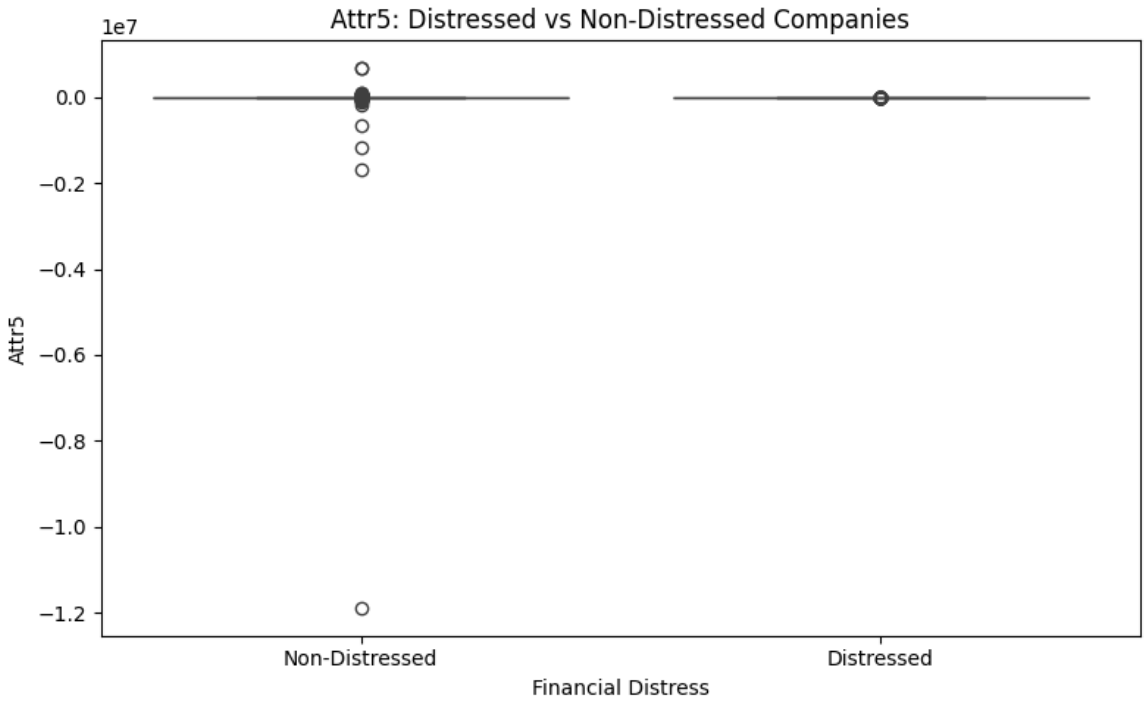
```
    plt.tight_layout()

    plt.show()
```

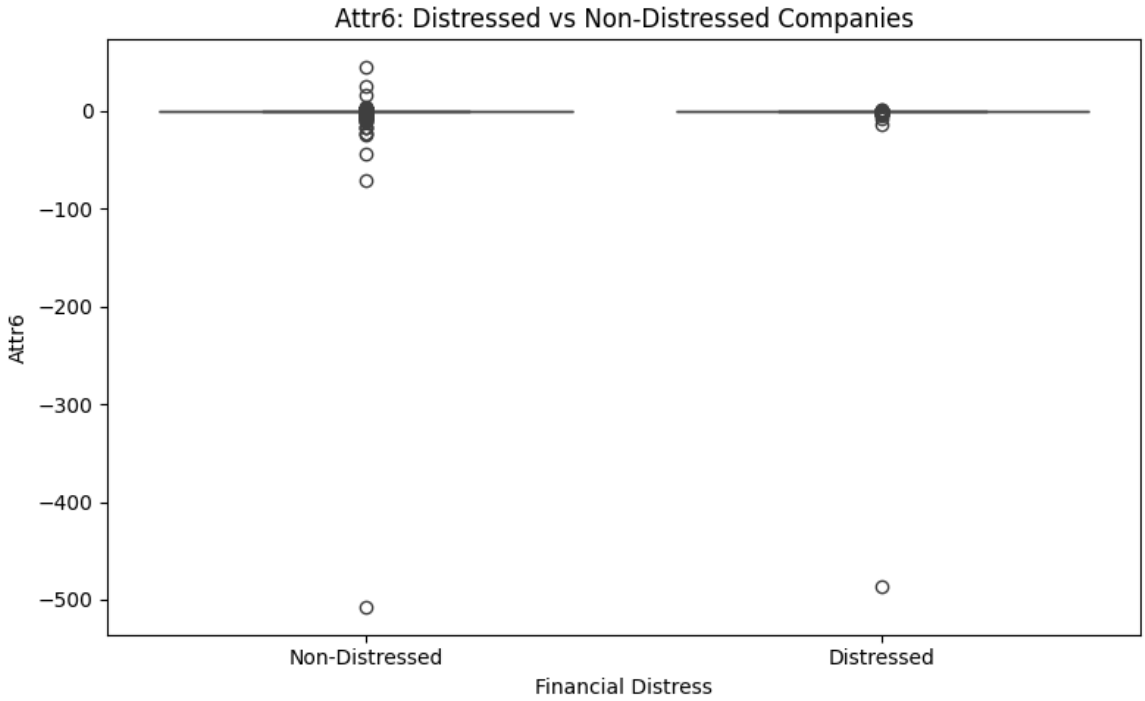





Attr3: Distressed vs Non-Distressed Companies
0
−100
−200
−300
−400
−500
Attr3
Non-Distressed
Distressed
Financial Distress

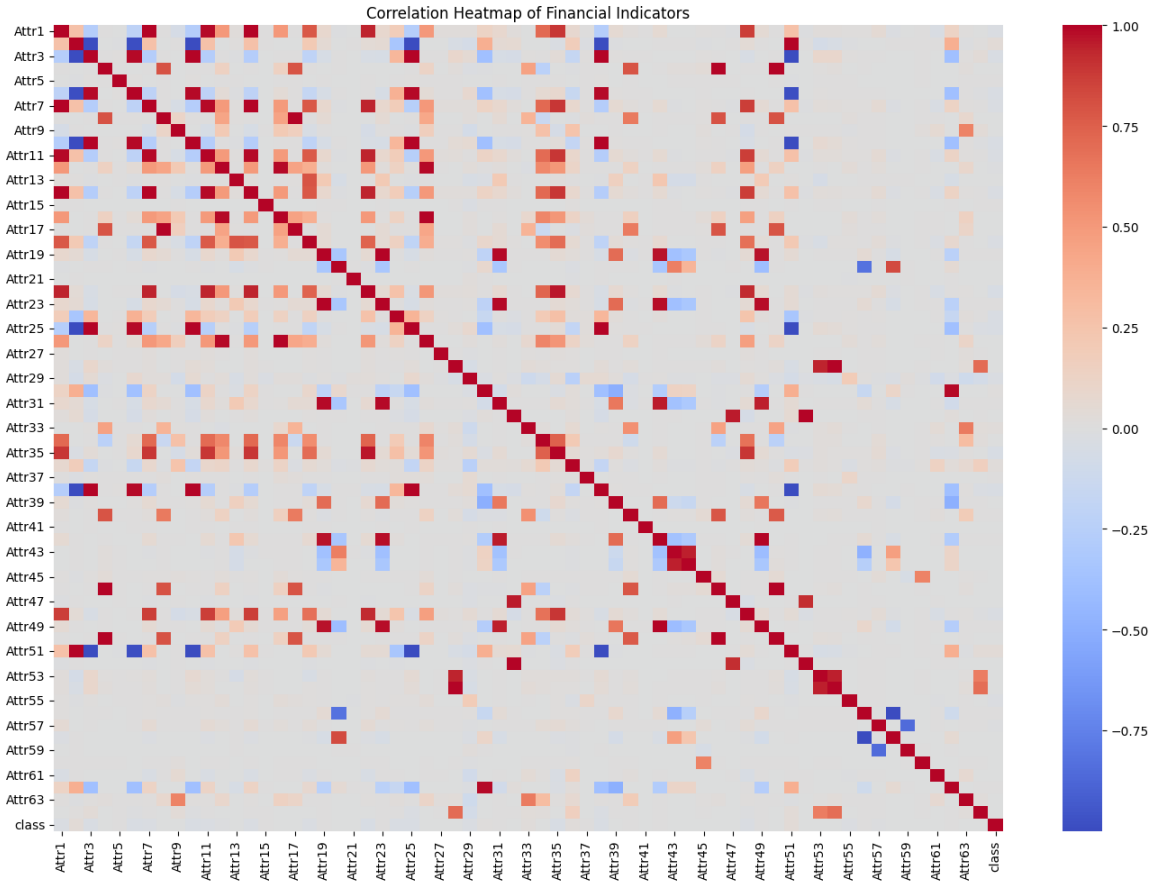

Attr4: Distressed vs Non-Distressed Companies
50000
40000
30000
20000
10000
0
Attr4
Non-Distressed
Distressed
Financial Distress





In [18]:
```
# ============================================================
# CORRELATION HEATMAP
# ============================================================

correlation = df.corr(
    numeric_only=True
)

plt.figure(figsize=(14, 10))

sns.heatmap(
    correlation,
    cmap="coolwarm",
    center=0
)
```

```
plt.title(
    "Correlation Heatmap of Financial Indicators"
)

plt.tight_layout()

plt.show()
```



In [19]:

```
# ============================================================
# CORRELATION WITH DISTRESS
# ============================================================

target_correlations = (
    correlation[target_column]
    .drop(target_column)
    .sort_values(
        key=abs,
        ascending=False
    )
)

print("=" * 70)
print("FINANCIAL INDICATORS MOST ASSOCIATED WITH DISTRESS")
print("=" * 70)

print(
    target_correlations.head(15)
)
```

```
================================================================
==
FINANCIAL INDICATORS MOST ASSOCIATED WITH DISTRESS
================================================================
==
Attr35   -0.044707
Attr22   -0.040441
Attr25   -0.036882
Attr10   -0.035791
Attr2     0.035597
Attr38   -0.035392
Attr3    -0.034326
Attr51    0.034185
Attr24   -0.033591
Attr6    -0.033115
Attr29   -0.029435
Attr14   -0.028549
Attr7    -0.028546
Attr1    -0.027098
Attr11   -0.027020
Name: class, dtype: float64
```

In [20]:
```
# ============================================================
# PREPARE FEATURES AND TARGET
# ============================================================

X = df.drop(
    columns=[target_column]
)

y = df[target_column]

print("Feature shape:", X.shape)

print("Target shape:", y.shape)

print("\nTarget distribution:")
print(y.value_counts())
```

```
Feature shape: (10416, 64)
Target shape: (10416,)

Target distribution:
class
0    9923
1     493
Name: count, dtype: int64
```

In [21]:
```
# ============================================================
# HANDLE INFINITE VALUES
# ============================================================

X = X.replace(
    [np.inf, -np.inf],
    np.nan
)
```

```
print(
    "Infinite values replaced successfully."
)

print(
    "Total missing values:",
    X.isnull().sum().sum()
)
```

```
Infinite values replaced successfully.
Total missing values: 9833
```

In [22]:
```
# ================================================================
# MISSING VALUE IMPUTATION
# ================================================================

imputer = SimpleImputer(
    strategy="median"
)

X_imputed = imputer.fit_transform(X)

X = pd.DataFrame(
    X_imputed,
    columns=X.columns,
    index=X.index
)

print(
    "Missing values after imputation:",
    X.isnull().sum().sum()
)
```

```
Missing values after imputation: 0
```

In [23]:
```
# ================================================================
# TRAIN TEST SPLIT
# ================================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))

print("Testing samples :", len(X_test))

print("\nTraining class distribution:")
print(y_train.value_counts())

print("\nTesting class distribution:")
print(y_test.value_counts())
```

```
Training samples: 8332
Testing samples : 2084

Training class distribution:
class
0    7938
1     394
Name: count, dtype: int64

Testing class distribution:
class
0    1985
1      99
Name: count, dtype: int64
```

In [24]:
```
# ============================================================
# DECISION TREE CLASSIFIER
# ============================================================

model = DecisionTreeClassifier(
    max_depth=8,
    min_samples_split=20,
    min_samples_leaf=10,
    class_weight="balanced",
    random_state=42
)

model.fit(
    X_train,
    y_train
)

print(
    "Decision Tree model trained successfully!"
)
```

```
Decision Tree model trained successfully!
```

In [25]:
```
# ============================================================
# MODEL PREDICTIONS
# ============================================================

y_pred = model.predict(
    X_test
)

print(
    "Predictions completed!"
)

print(
    "\nFirst 20 predictions:"
)

print(
    y_pred[:20]
)
```

```
Predictions completed!

First 20 predictions:
[1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0]
```

In [26]:

```python
# ============================================================
# CLASSIFICATION METRICS
# ============================================================

accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print("=" * 70)
print("DECISION TREE MODEL PERFORMANCE")
print("=" * 70)

print(
    f"Accuracy  : {accuracy:.4f}"
)

print(
    f"Precision : {precision:.4f}"
)

print(
    f"Recall    : {recall:.4f}"
)

print(
    f"F1 Score  : {f1:.4f}"
)
```

```
==========================================================================
==
DECISION TREE MODEL PERFORMANCE
==========================================================================
==
Accuracy  : 0.8335
Precision : 0.1915
Recall    : 0.7778
F1 Score  : 0.3074
```

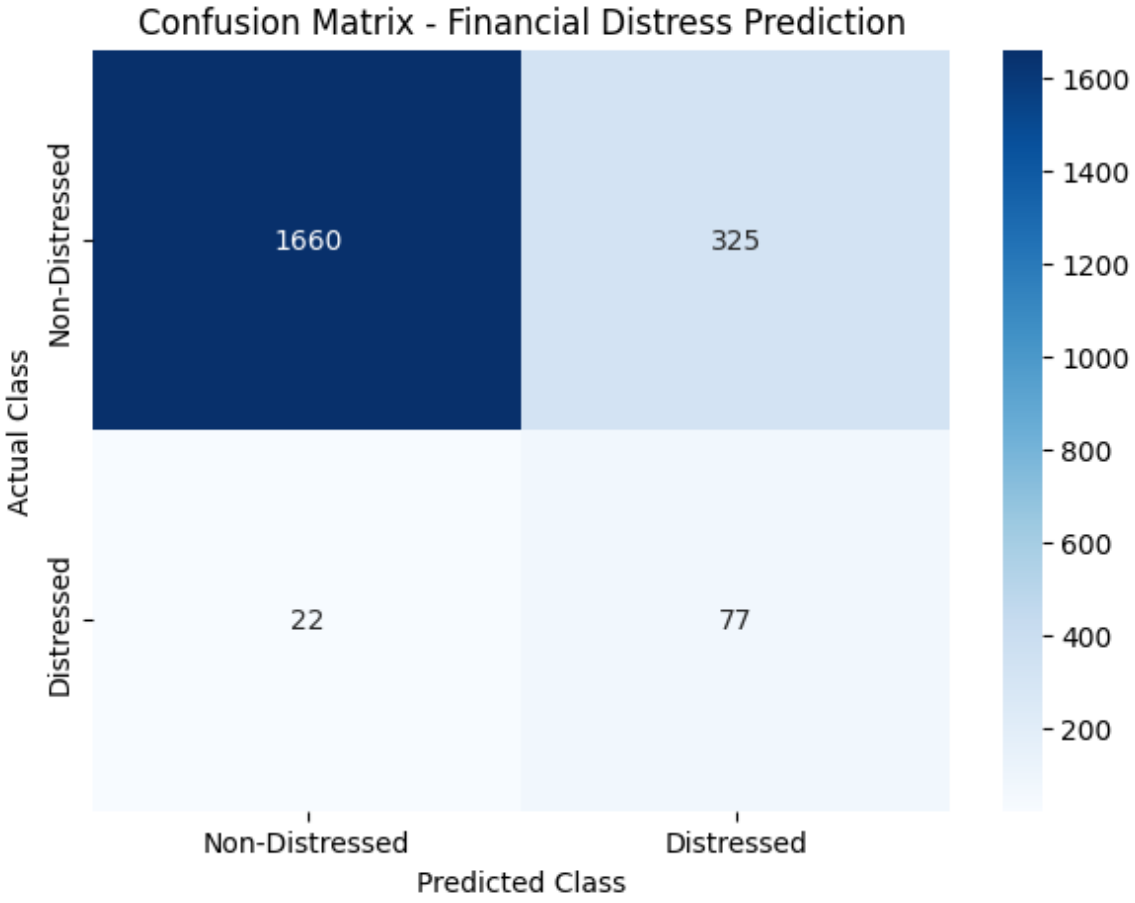
In [27]:
```
# ============================================================
# CLASSIFICATION REPORT
# ============================================================

print(
    classification_report(
        y_test,
        y_pred,
        target_names=[
            "Non-Distressed",
            "Distressed"
        ],
        zero_division=0
    )
)
```

```
                precision    recall  f1-score   support

Non-Distressed       0.99      0.84      0.91      1985
    Distressed       0.19      0.78      0.31        99

      accuracy                           0.83      2084
     macro avg       0.59      0.81      0.61      2084
  weighted avg       0.95      0.83      0.88      2084
```

In [28]:
```
# ============================================================
# CONFUSION MATRIX
# ============================================================

cm = confusion_matrix(
    y_test,
    y_pred
)

print("Confusion Matrix:")
print(cm)

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=[
        "Non-Distressed",
```

```
        "Distressed"
    ],
    yticklabels=[
        "Non-Distressed",
        "Distressed"
    ]
)

plt.title(
    "Confusion Matrix - Financial Distress Prediction"
)

plt.xlabel(
    "Predicted Class"
)

plt.ylabel(
    "Actual Class"
)

plt.show()
```

```
Confusion Matrix:
[[1660  325]
 [  22   77]]
```



In [29]:
```
# ============================================================
# FEATURE IMPORTANCE
# ============================================================

feature_importance = pd.DataFrame({
```

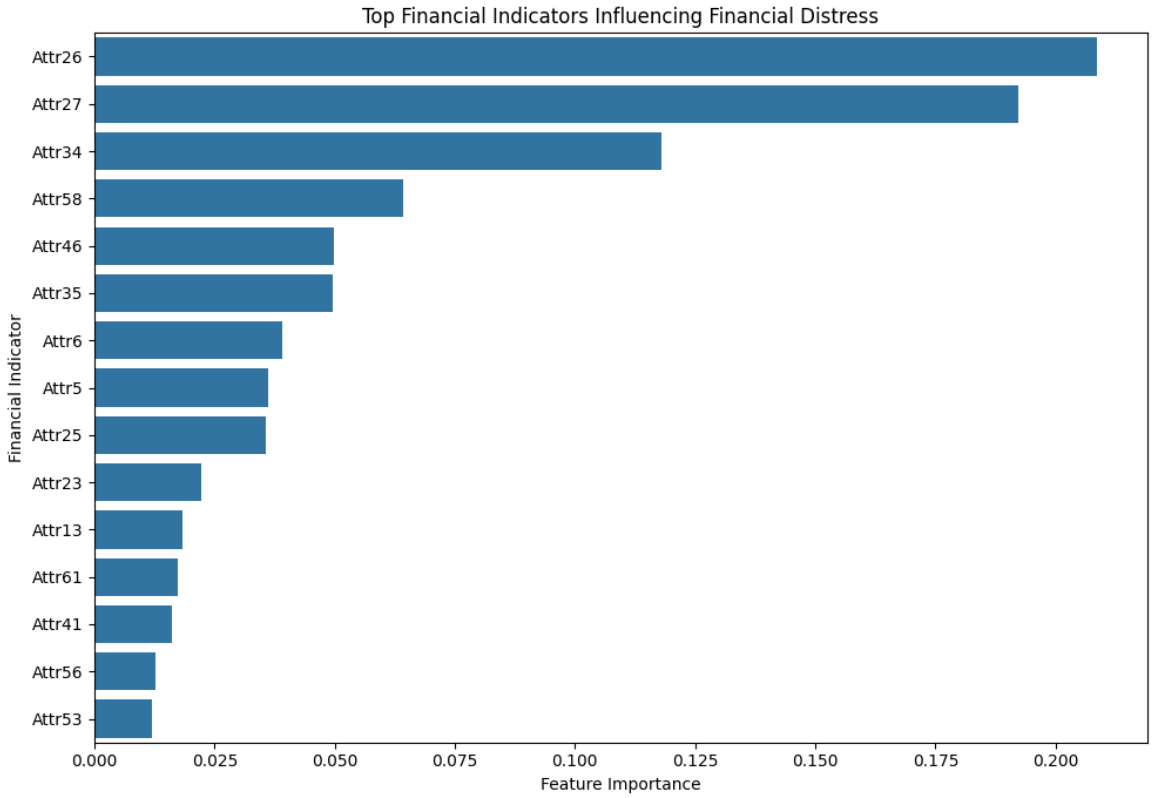
```
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

print("=" * 70)
print("TOP FINANCIAL INDICATORS ASSOCIATED WITH DISTRESS")
print("=" * 70)

print(
    feature_importance.head(20).to_string(
        index=False
    )
)
```

```
======================================================================
==
TOP FINANCIAL INDICATORS ASSOCIATED WITH DISTRESS
======================================================================
==
Feature  Importance
 Attr26    0.208689
 Attr27    0.192326
 Attr34    0.118084
 Attr58    0.064138
 Attr46    0.049829
 Attr35    0.049586
  Attr6    0.039155
  Attr5    0.036234
 Attr25    0.035598
 Attr23    0.022312
 Attr13    0.018368
 Attr61    0.017319
 Attr41    0.016235
 Attr56    0.012620
 Attr53    0.012006
 Attr21    0.011571
  Attr3    0.011472
  Attr7    0.010035
 Attr39    0.008838
 Attr47    0.008414
```

In [30]:

```
# ============================================================
# FEATURE IMPORTANCE VISUALIZATION
# ============================================================

top_features = feature_importance.head(15)

plt.figure(figsize=(10, 7))

sns.barplot(
    data=top_features,
```

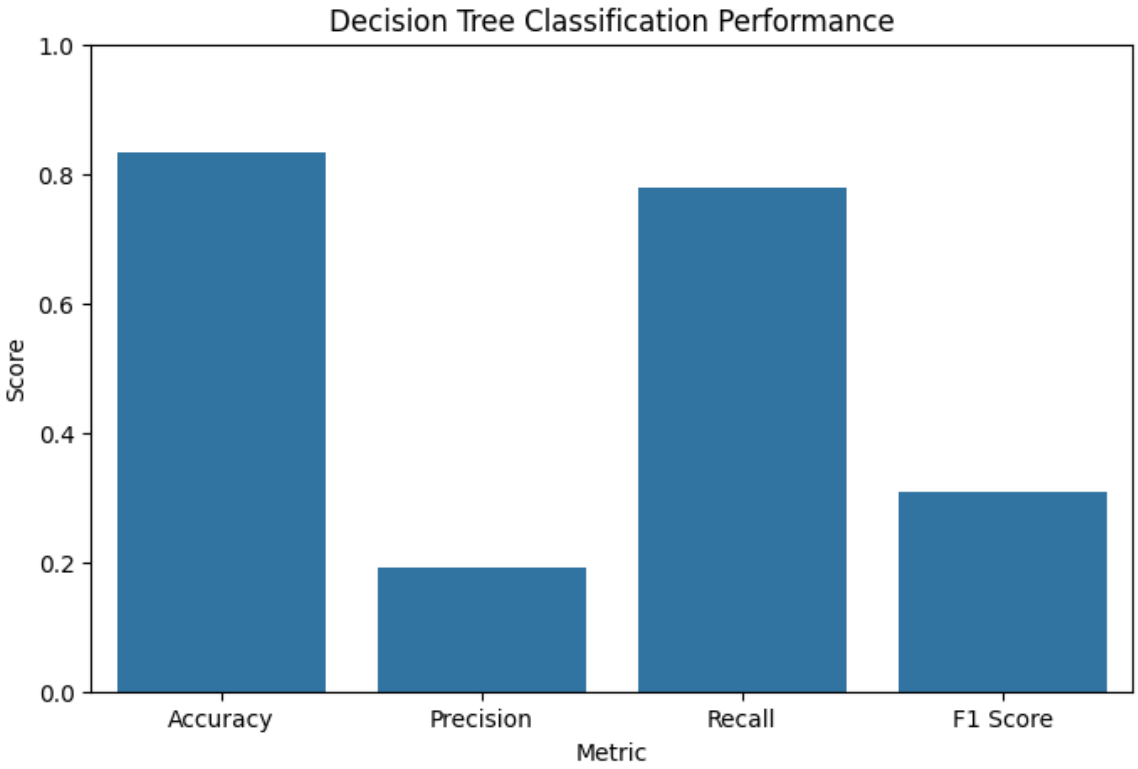
```
    x="Importance",
    y="Feature"
)

plt.title(
    "Top Financial Indicators Influencing Financial Distress"
)

plt.xlabel(
    "Feature Importance"
)

plt.ylabel(
    "Financial Indicator"
)

plt.tight_layout()

plt.show()
```



In [31]:

```
# ============================================================
# MODEL PERFORMANCE VISUALIZATION
# ============================================================

metrics = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ],

    "Score": [
```

```
        accuracy,
        precision,
        recall,
        f1
    ]
})

plt.figure(figsize=(8, 5))

sns.barplot(
    data=metrics,
    x="Metric",
    y="Score"
)

plt.ylim(0, 1)

plt.title(
    "Decision Tree Classification Performance"
)

plt.ylabel(
    "Score"
)

plt.show()
```



In [32]:
```
# ============================================================
# TOP INDICATOR DISTRIBUTIONS
# ============================================================

top_5_features = (
    feature_importance
    .head(5)["Feature"]
```

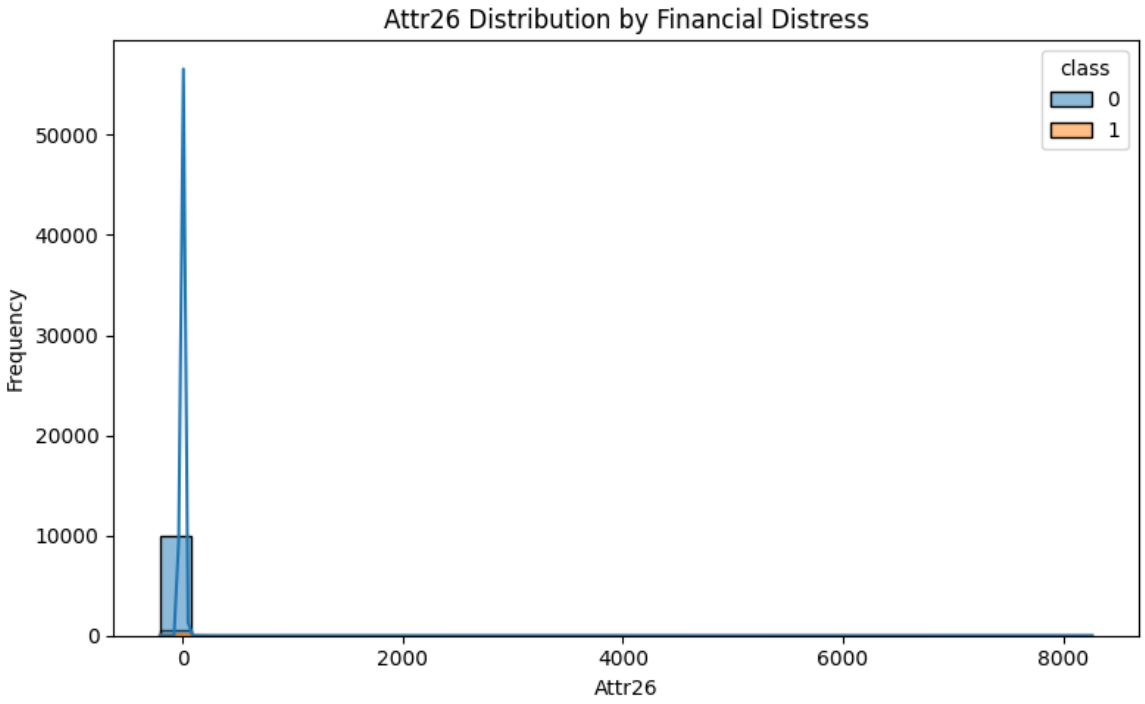
```
    .tolist()
)

print(
    "Top 5 financial indicators:"
)

print(top_5_features)
```

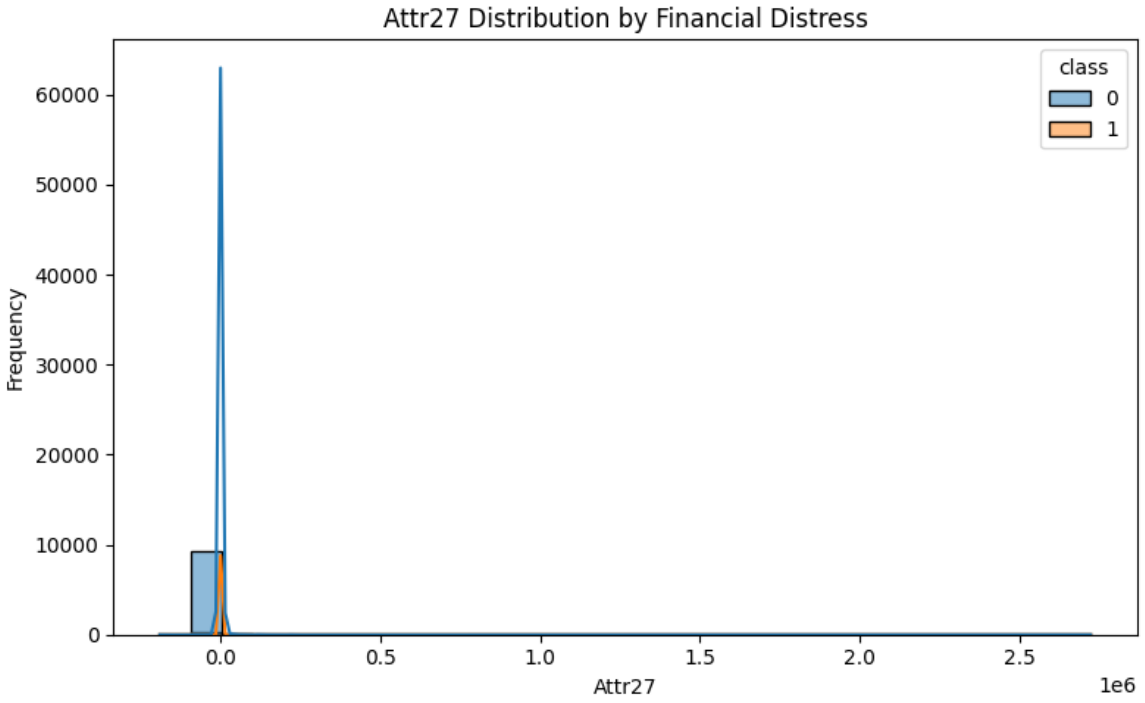
```
Top 5 financial indicators:
['Attr26', 'Attr27', 'Attr34', 'Attr58', 'Attr46']
```

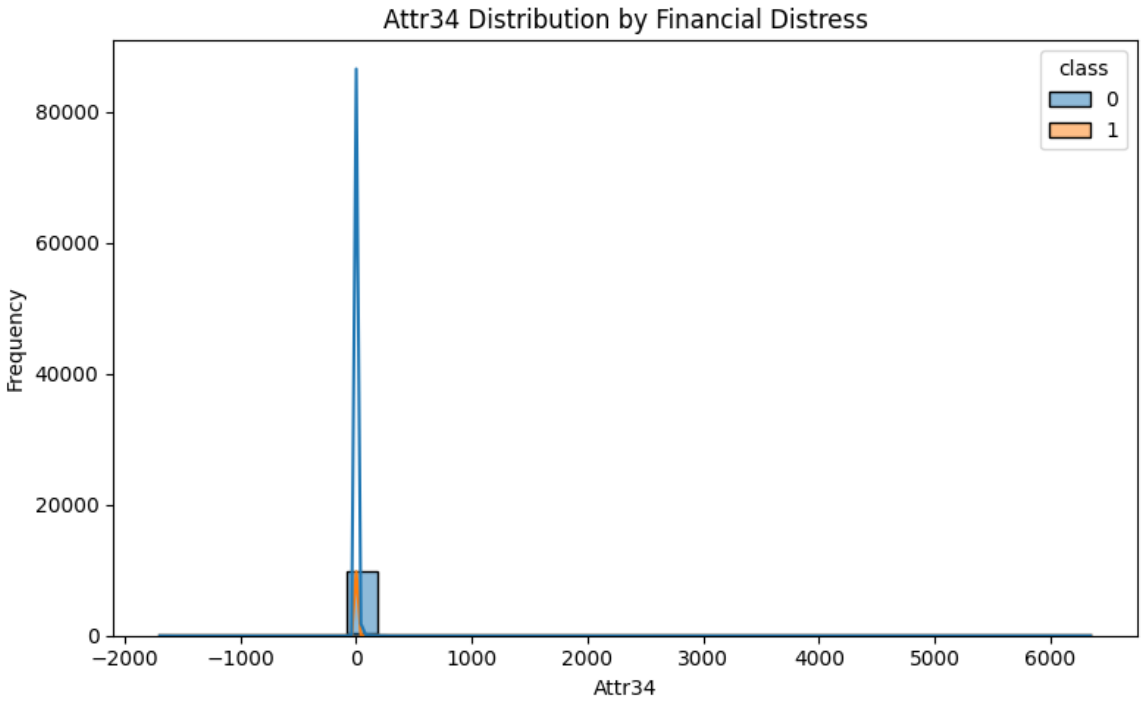
In [33]:
```
# ============================================================
# TOP INDICATOR DISTRIBUTIONS
# ============================================================

top_5_features = (
    feature_importance
    .head(5)["Feature"]
    .tolist()
)

print(
    "Top 5 financial indicators:"
)

print(top_5_features)
```

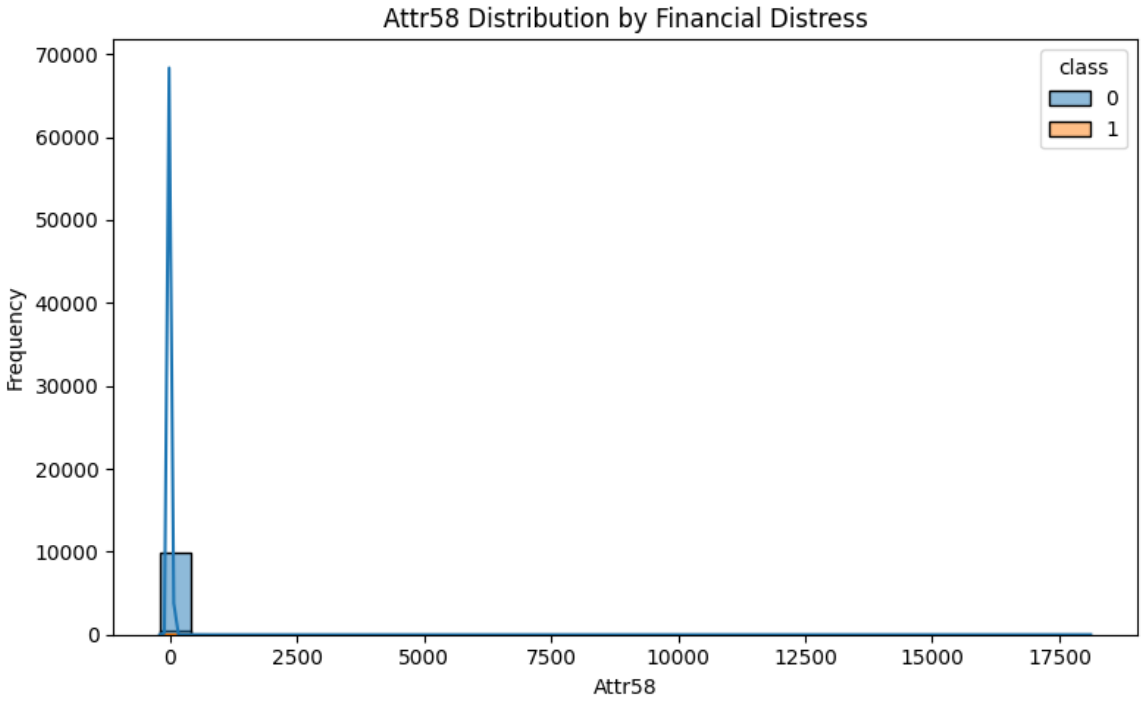
```
Top 5 financial indicators:
['Attr26', 'Attr27', 'Attr34', 'Attr58', 'Attr46']
```

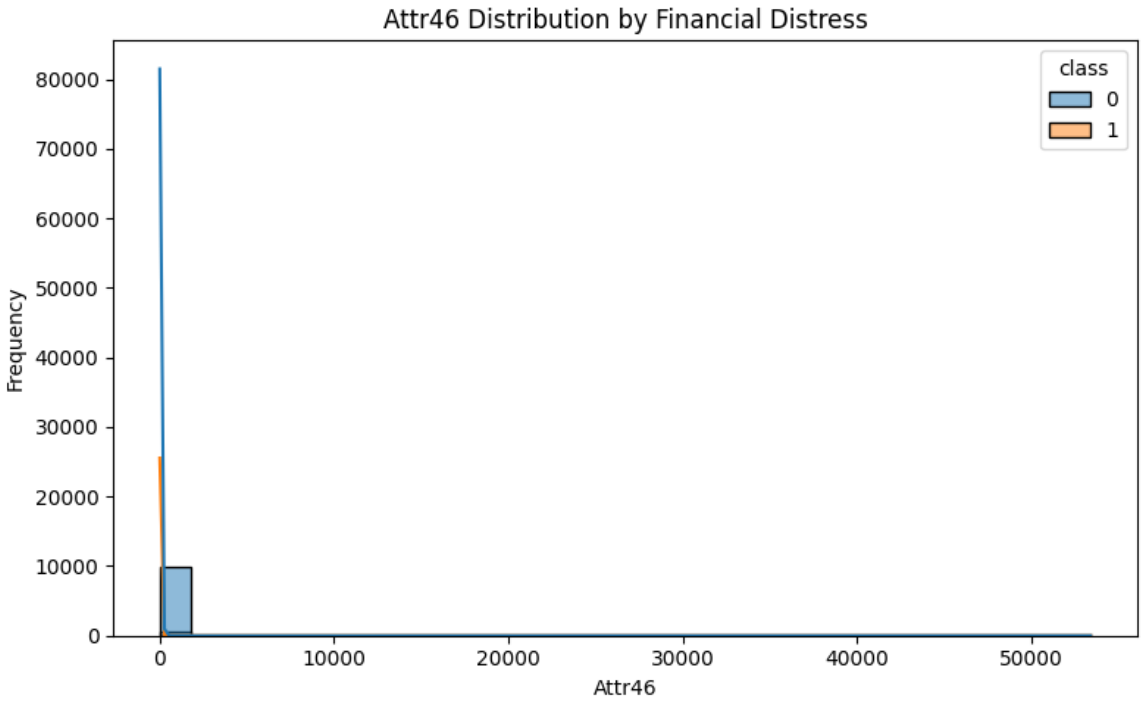
In [34]:
```
for feature in top_5_features:

    plt.figure(figsize=(8, 5))

    sns.histplot(
        data=df,
        x=feature,
        hue=target_column,
        kde=True,
        bins=30
    )

    plt.title(
        f"{feature} Distribution by Financial Distress"
    )

    plt.xlabel(feature)

    plt.ylabel("Frequency")

    plt.tight_layout()

    plt.show()
```


Attr26 Distribution by Financial Distress
class
0
1
Frequency
50000
40000
30000
20000
10000
0
0
2000
4000
6000
8000
Attr26

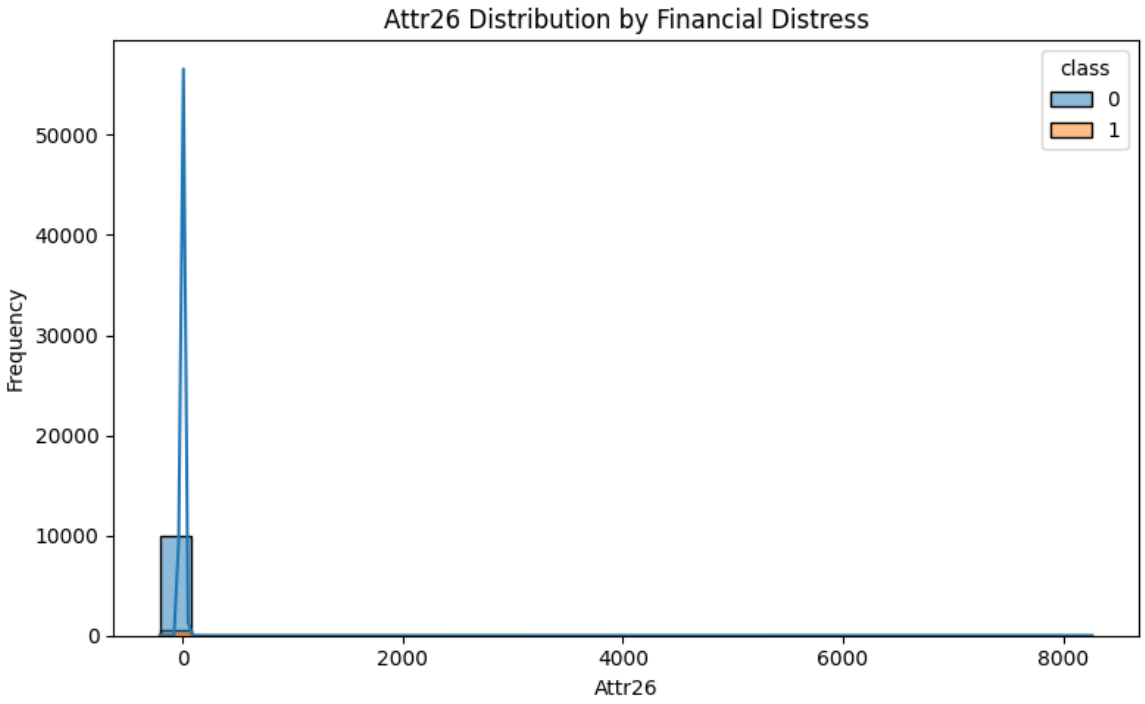

Attr27 Distribution by Financial Distress
class
0
1
Frequency
60000
50000
40000
30000
20000
10000
0
0.0
0.5
1.0
1.5
2.0
2.5
Attr27
1e6

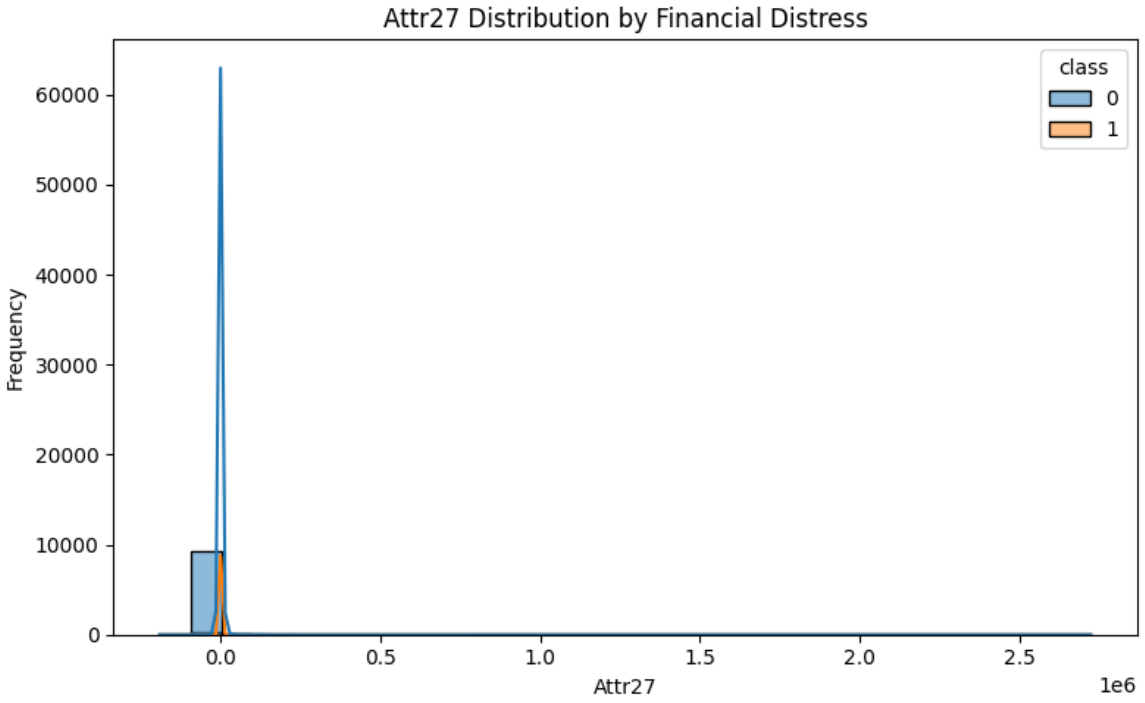

Attr34 Distribution by Financial Distress
class
0
1
Frequency
80000
60000
40000
20000
0
−2000
−1000
0
1000
2000
3000
4000
5000
6000
Attr34

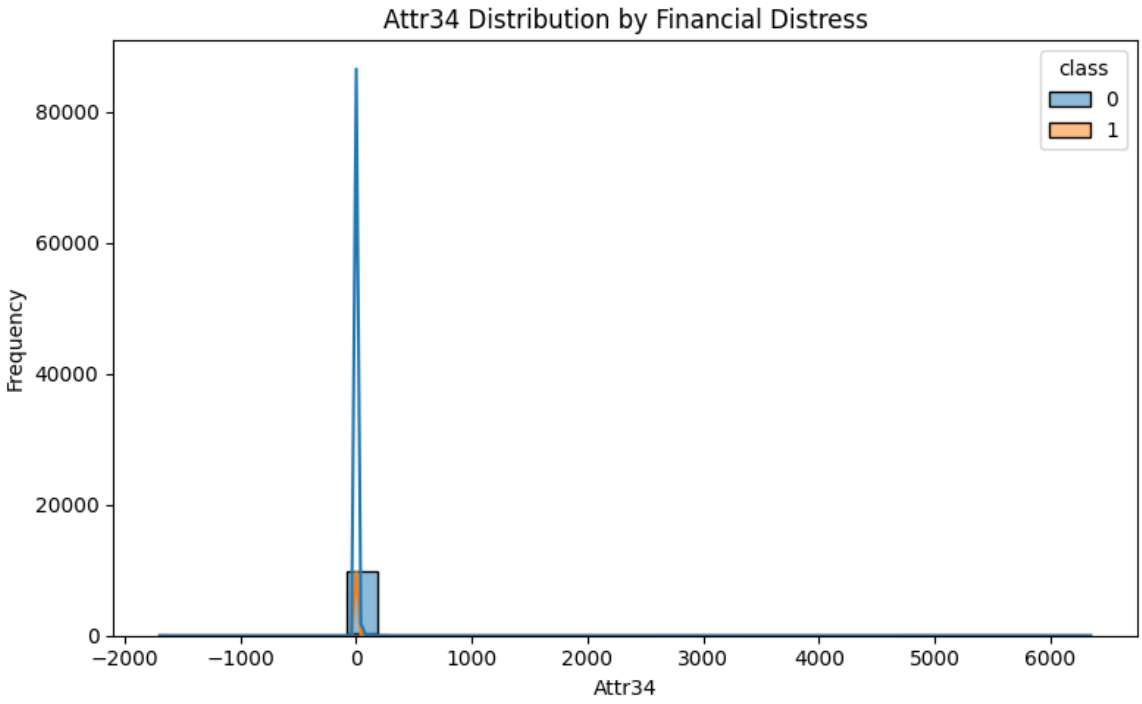

Attr58 Distribution by Financial Distress
class
0
1
Frequency
70000
60000
50000
40000
30000
20000
10000
0
0
2500
5000
7500
10000
12500
15000
17500
Attr58

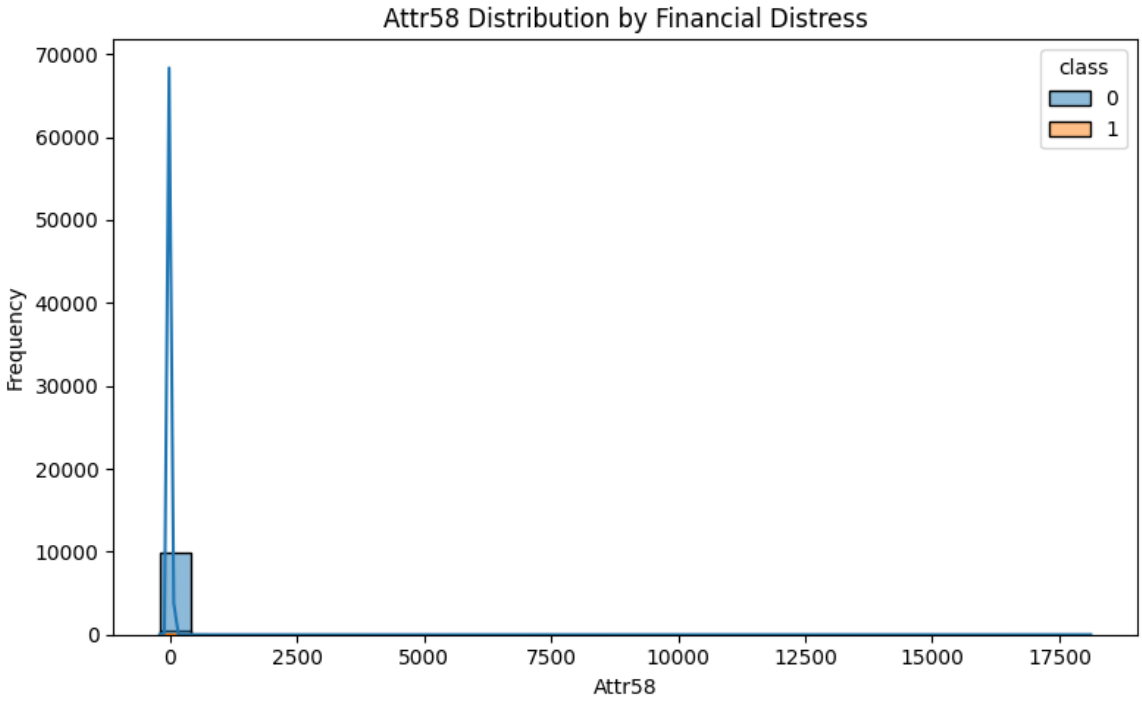


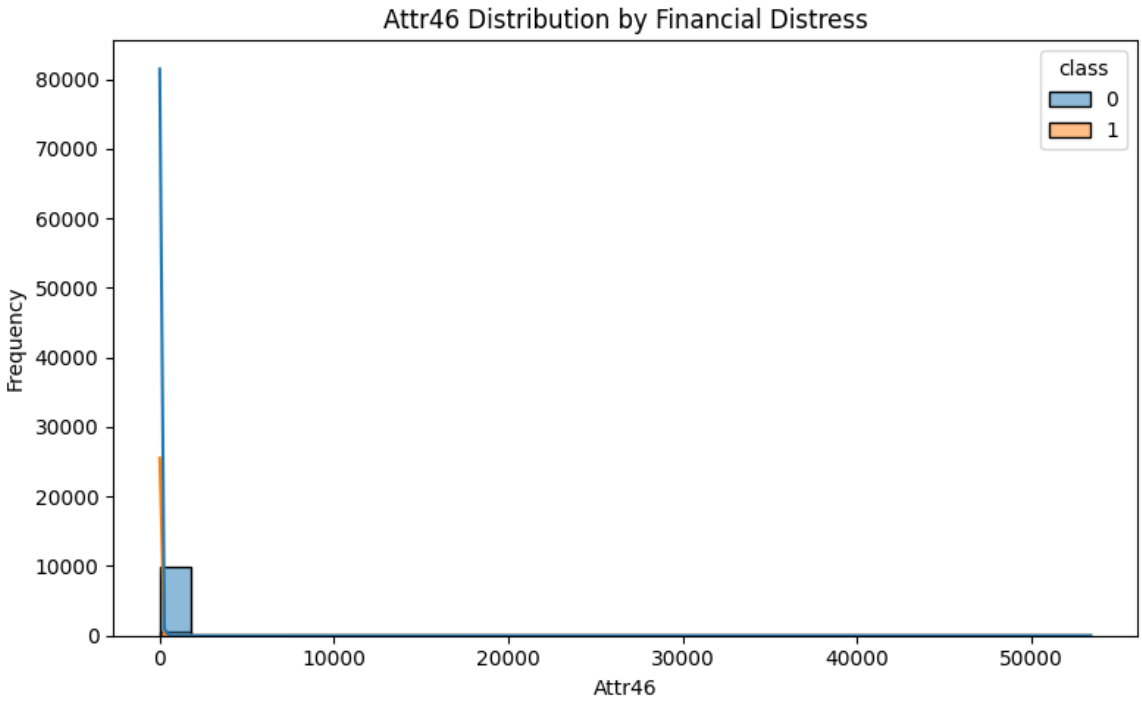
In [35]:
```
for feature in top_5_features:

    plt.figure(figsize=(8, 5))

    sns.histplot(
        data=df,
        x=feature,
        hue=target_column,
        kde=True,
        bins=30
    )

    plt.title(
        f"{feature} Distribution by Financial Distress"
    )

    plt.xlabel(feature)

    plt.ylabel("Frequency")

    plt.tight_layout()

    plt.show()
```


Attr26 Distribution by Financial Distress
class
0
1
Frequency
50000
40000
30000
20000
10000
0
0
2000
4000
6000
8000
Attr26


Attr27 Distribution by Financial Distress
class
0
1
Frequency
60000
50000
40000
30000
20000
10000
0
0.0
0.5
1.0
1.5
2.0
2.5
Attr27
1e6


Attr34 Distribution by Financial Distress
class
0
1
Frequency
80000
60000
40000
20000
0
−2000
−1000
0
1000
2000
3000
4000
5000
6000
Attr34


Attr58 Distribution by Financial Distress
class
0
1
Frequency
70000
60000
50000
40000
30000
20000
10000
0
0
2500
5000
7500
10000
12500
15000
17500
Attr58



In [36]:
```
# ============================================================
# FINAL RESULTS TABLE
# ============================================================

results = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ],

    "Score": [
        accuracy,
        precision,
        recall,
        f1
    ]
})

results["Score"] = results["Score"].round(4)

print("=" * 60)
print("FINAL MODEL RESULTS")
print("=" * 60)

print(results)
```

```
============================================================
FINAL MODEL RESULTS
============================================================
      Metric   Score
0   Accuracy  0.8335
1  Precision  0.1915
2     Recall  0.7778
3   F1 Score  0.3074
```

In [37]:
```python
# ============================================================
# FINAL PROJECT SUMMARY
# ============================================================

print("=" * 75)
print("CORPORATE FINANCIAL DISTRESS PREDICTION")
print("=" * 75)

print(
    "\nTotal companies analyzed:",
    len(df)
)

print(
    "Number of financial indicators:",
    X.shape[1]
)

print(
    "\nNon-distressed companies:",
    (y == 0).sum()
)

print(
    "Distressed companies:",
    (y == 1).sum()
)

print(
    "\nDecision Tree Accuracy:",
    round(accuracy * 100, 2),
    "%"
)

print(
    "Decision Tree Precision:",
    round(precision * 100, 2),
    "%"
)

print(
    "Decision Tree Recall:",
    round(recall * 100, 2),
    "%"
)

print(
```

```
    "Decision Tree F1 Score:",
    round(f1 * 100, 2),
    "%"
)

print("\nTop 10 Financial Indicators:")

print(
    feature_importance
    .head(10)
    .to_string(index=False)
)

print("\nProject completed successfully!")
```

```
==========================================================================
=======
CORPORATE FINANCIAL DISTRESS PREDICTION
==========================================================================
=======

Total companies analyzed: 10416
Number of financial indicators: 64

Non-distressed companies: 9923
Distressed companies: 493

Decision Tree Accuracy: 83.35 %
Decision Tree Precision: 19.15 %
Decision Tree Recall: 77.78 %
Decision Tree F1 Score: 30.74 %

Top 10 Financial Indicators:
Feature  Importance
 Attr26    0.208689
 Attr27    0.192326
 Attr34    0.118084
 Attr58    0.064138
 Attr46    0.049829
 Attr35    0.049586
  Attr6    0.039155
  Attr5    0.036234
 Attr25    0.035598
 Attr23    0.022312

Project completed successfully!
```

In [38]:
```
# ============================================================
# SAVE FEATURE IMPORTANCE AND RESULTS
# ============================================================

feature_importance.to_csv(
    "financial_distress_feature_importance.csv",
    index=False
)
```

```
results.to_csv(
    "financial_distress_model_results.csv",
    index=False
)

print("Results saved successfully!")
```

```
Results saved successfully!
```

In [39]:
```
import nbformat

# Your original notebook
input_file = "padv.ipynb"

# New notebook containing only outputs
output_file = "outputs_only.ipynb"

# Read notebook
with open(input_file, "r", encoding="utf-8") as f:
    nb = nbformat.read(f, as_version=4)

# Remove code but keep outputs
for cell in nb.cells:
    if cell.cell_type == "code":
        cell.source = ""

# Save as a new .ipynb file
with open(output_file, "w", encoding="utf-8") as f:
    nbformat.write(nb, f)

print("Created:", output_file)
```

```
Created: outputs_only.ipynb
```

In [ ]:
```

```